

PRACTICAL WEEK10

TASK1: Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
rishill10@ASUS16X: ~/Practica × + ▾
stat original.txt
Hello Ubuntu
50825 original.txt
  File: original.txt
  size: 13          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 50825        Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/rishill10)   Gid: ( 1000/rishill10)
Access: 2026-09-26 06:12:34.167083336 +0000
Modify: 2026-09-26 06:12:34.159083337 +0000
Change: 2026-09-26 06:12:34.159083337 +0000
Birth: 2026-09-26 06:12:34.159083337 +0000
rishill10@ASUS16X:~/Practical10$ ln original.txt hardlink.txt
ls -li original.txt hardlink.txt
50825 hardlink.txt  50825 original.txt
rishill10@ASUS16X:~/Practical10$ echo "New line" >> hardlink.txt
cat original.txt
Hello Ubuntu
New line
rishill10@ASUS16X:~/Practical10$ ln -s original.txt symlink.txt
ls -li
stat symlink.txt
50825 hardlink.txt  50825 original.txt  50845 symlink.txt
  File: 'symlink.txt' -> 'original.txt'
  size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48      Inode: 50845        Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/rishill10)   Gid: ( 1000/rishill10)
Access: 2026-09-26 06:12:51.071077411 +0000
Modify: 2026-09-26 06:12:51.059077416 +0000
Change: 2026-09-26 06:12:51.059077416 +0000
Birth: 2026-09-26 06:12:51.059077416 +0000
```

```
rishill10@ASUS16X:~/Practical10$ stat -L symlink.txt
  File: symlink.txt
  size: 22          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 50825         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/rishill10)   Gid: ( 1000/rishill10)
Access: 2026-09-26 06:12:44.819079599 +0000
Modify: 2026-09-26 06:12:44.811079602 +0000
Change: 2026-09-26 06:12:44.811079602 +0000
 Birth: 2026-09-26 06:12:34.159083337 +0000
rishill10@ASUS16X:~/Practical10$ find . -inum 421212
rishill10@ASUS16X:~/Practical10$ rm original.txt
ls -li
cat symlink.txt
50825 hardlink.txt 50845 symlink.txt
cat: symlink.txt: No such file or directory
rishill10@ASUS16X:~/Practical10$ |
```

TASK2: Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```
GNU nano 8.7.1 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main()
{
    const char *filename = "mmap_data.txt";
    const char *message = "Hello from mmap!";

    int fd;
    struct stat st;
    char *mapped;

    fd = open(filename, O_RDWR | O_CREAT, 0666);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    if (ftruncate(fd, 100) == -1)
    {
        perror("ftruncate");
        close(fd);
        return 1;
    }

    if (fstat(fd, &st) == -1)
    {
        perror("fstat");
        close(fd);
        return 1;
    }

    mapped = mmap(NULL, st.st_size, PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);
```

```

    if (mapped == MAP_FAILED)
    {
        perror("mmap");
        close(fd);
        return 1;
    }

    strcpy(mapped, message);

    if (msync(mapped, st.st_size, MS_SYNC) == -1)
    {
        perror("msync");
    }

    printf("Data written using mmap(): %s\n", mapped);
    printf("Data read using mmap(): %s\n", mapped);

    if (munmap(mapped, st.st_size) == -1)
    {
        perror("munmap");
    }

    close(fd);
    return 0;
}

```

```

rishill10@ASUS16X: ~/Practica × + ▾
rishill10@ASUS16X:~/Practical10$ nano task2.c
rishill10@ASUS16X:~/Practical10$ gcc task2.c -o task2
./task2
Data written using mmap(): Hello from mmap!
Data read using mmap(): Hello from mmap!
rishill10@ASUS16X:~/Practical10$ cat mmap_data.txt
Hello from mmap!rishill10@ASUS16X:~/Practical10$ |

```